

Heavens Light is Our Guide



**Rajshahi University of Engineering and
Technology**
**Department of Electrical & Computer
Engineering**

Course Title

Digital Techniques Sessional

Course No: ECE 2112

Lab Report -01

Name of the Experiment: Introduction to Digital Logic

Date of Submission: 25 July 2026

Submitted By:

Uddipon Chowdhury
Roll: 2410019
Registration No: 1072

Submitted To:

Mst. Mazeda Noor Tasnim
Lecturer
ECE, RUET

1 Experiment 1: Implementation of OR Gate Using NOR Gates

1.1 Objective

The objective of this experiment is to design and simulate an OR gate using only NOR gates in Logisim Evolution and verify its operation using the corresponding truth table.

1.2 Theory

Digital logic circuits are constructed using logic gates, which perform fundamental Boolean operations on binary inputs. Among the various logic gates, the OR gate and NOR gate are widely used in digital systems.

An OR gate produces a logic HIGH (1) whenever at least one of its inputs is HIGH. The Boolean expression of a two-input OR gate is

$$Y = A + B \quad (1)$$

where A and B are the input variables and Y is the output. The truth table of a two-input OR gate is shown in Table 5.

Table 1: Truth Table of a Two-Input OR Gate

A	B	$Y = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

A NOR gate is formed by connecting a NOT gate after an OR gate. Therefore, its output is the complement of the OR operation. The Boolean expression of a two-input NOR gate is

$$Y = \overline{A + B} \quad (2)$$

The truth table of the NOR gate is presented in Table 2.

Table 2: Truth Table of a Two-Input NOR Gate

A	B	$Y = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

The NOR gate is known as a **universal gate** because any logic gate can be implemented using only NOR gates. To realize an OR gate using NOR gates, the output of the first NOR gate is inverted using a second NOR gate whose two inputs are connected together.

Mathematically,

$$X = \overline{A + B} \quad (3)$$

$$Y = \overline{X + X} \quad (4)$$

$$= \overline{\overline{A + B} + \overline{A + B}} \quad (5)$$

$$= A + B \quad (6)$$

Thus, the OR function is successfully obtained using only NOR gates.

1.3 Circuit Diagram

The OR gate was implemented using two NOR gates in Logisim Evolution. The completed circuit is shown in Figure 1.

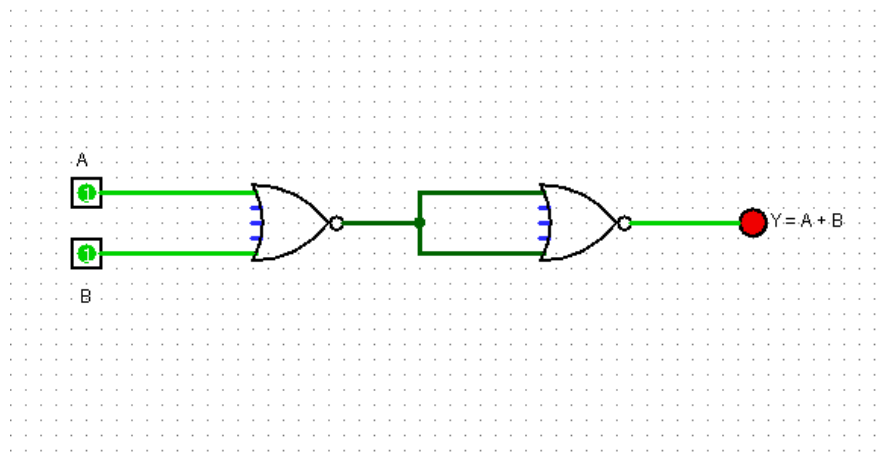


Figure 1: Implementation of an OR gate using NOR gates in Logisim Evolution.

1.4 Procedure

The experiment was carried out in Logisim according to the following steps:

1. Open the Logisim software and create a new project.
2. From the **Gates** library, place two **NOR** gates onto the workspace.
3. Connect two input pins, labeled A and B , to the inputs of the first NOR gate.
4. Connect the output of the first NOR gate to both input terminals of the second NOR gate so that it functions as an inverter.
5. Attach an output pin to the output of the second NOR gate.
6. Verify that all connections are properly completed and that the circuit has no wiring errors.
7. Apply all possible combinations of binary inputs (00, 01, 10, 11) by toggling the input pins.
8. Observe the output corresponding to each input combination and record the results.
9. Compare the observed outputs with the theoretical truth table of the OR gate to verify the correctness of the implementation.

1.5 Observation Table

Table 3: Observed Truth Table			
A	B	Expected Output	Observed Output
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

1.6 Discussion

The OR gate was successfully realized using only NOR gates in Logisim Evolution. The circuit was designed based on the Boolean expression of the OR function using universal gates. The simulation was performed for all possible input combinations, and the observed outputs were found to

be identical to the theoretical truth table of the OR gate. This experiment demonstrates that the NOR gate can be used as a universal gate to implement other logic functions and reinforces the concepts of Boolean algebra and digital logic design.

1.7 Result

The OR gate was successfully implemented using only NOR gates in Logisim Evolution. The simulation results matched the theoretical truth table for all input combinations, confirming the correctness of the circuit.

1.8 Conclusion

In this experiment, an OR gate was implemented and verified using only NOR gates in Logisim Evolution. The simulation results agreed with the theoretical analysis for all input combinations. The experiment confirmed the universality of the NOR gate and enhanced the understanding of logic gate implementation using Boolean algebra.

2 Experiment 2: Implementation of OR Gate Using NAND Gates

2.1 Objective

The objective of this experiment is to:

- Understand the operation of the universal NAND gate.
- Implement an OR gate using only NAND gates based on De Morgan's theorem.
- Design and simulate the circuit using **Logisim Evolution**.
- Verify the correctness of the implemented circuit by comparing the simulation results with the standard OR gate truth table.

2.2 Theory

A NAND (NOT-AND) gate is one of the most important logic gates in digital electronics because it is classified as a **universal gate**. A universal gate is capable of implementing any Boolean function without requiring any other

type of logic gate. Consequently, all basic logic gates such as AND, OR, NOT, XOR, and XNOR can be realized solely by combining NAND gates.

The Boolean expression of a two-input NAND gate is

$$Y = \overline{A \cdot B} \quad (7)$$

where A and B are the inputs and Y is the output.

According to **De Morgan's theorem**,

$$A + B = \overline{\overline{A} \cdot \overline{B}} \quad (8)$$

The complemented inputs can easily be obtained using NAND gates because connecting both inputs of a NAND gate together produces an inverter.

$$\overline{A} = A \text{ NAND } A \quad (9)$$

$$\overline{B} = B \text{ NAND } B \quad (10)$$

Therefore, the OR gate can be implemented using only NAND gates as

$$A + B = (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B) \quad (11)$$

This implementation requires three NAND gates:

1. One NAND gate to invert input A .
2. One NAND gate to invert input B .
3. One NAND gate to combine the inverted inputs and produce the OR output.

2.3 Required Software

- Logisim Evolution

2.4 Circuit Diagram

Figure 2 shows the implementation of an OR gate using only NAND gates designed in Logisim Evolution.

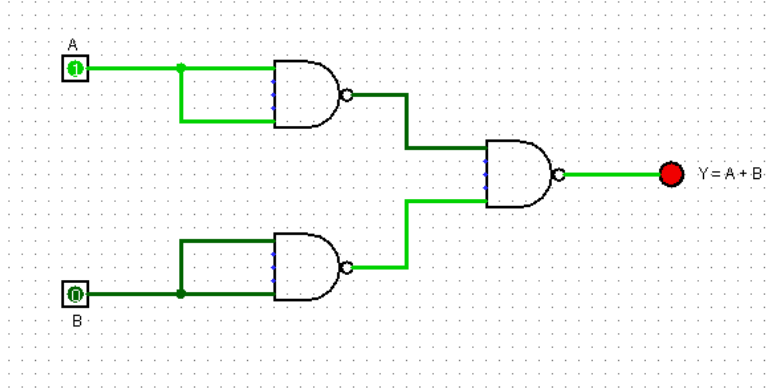


Figure 2: Implementation of OR Gate Using NAND Gates in Logisim Evolution

2.5 Truth Table of NAND Gate

The truth table of a two-input NAND gate is shown in Table 4.

Table 4: Truth Table of NAND Gate

A	B	$Y = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

2.6 Simulation Procedure

The experiment was carried out entirely through digital simulation using **Logisim Evolution**. No physical hardware implementation was performed.

1. Open Logisim Evolution and create a new project.
2. Place three two-input NAND gates from the Gates library.
3. Configure the first NAND gate by connecting both of its inputs to input *A* so that it functions as a NOT gate.
4. Configure the second NAND gate similarly by connecting both of its inputs to input *B*.
5. Connect the outputs of the first and second NAND gates to the inputs of the third NAND gate.

6. Connect two input pins labeled A and B .
7. Connect one output pin labeled Y to the output of the third NAND gate.
8. Apply all possible combinations of binary inputs using the input switches.
9. Observe the output for each input combination.
10. Compare the obtained output with the standard truth table of the OR gate.
11. Save the completed circuit and capture the simulation screenshot.

2.7 Simulation Setup

The complete circuit was designed and verified using Logisim Evolution. The simulation environment consisted of:

- Two input switches representing logic inputs A and B .
- Three two-input NAND gates.
- One output indicator (LED/output pin).
- Real-time logic simulation in Logisim Evolution.

2.8 Observed Truth Table

The observed outputs obtained from the Logisim Evolution simulation are presented in Table 5.

Table 5: Observed Truth Table of OR Gate Using NAND Gates

A	B	Output ($A + B$)
0	0	0
0	1	1
1	0	1
1	1	1

2.9 Result

The simulation output exactly matched the expected truth table of the OR gate for all possible input combinations. This confirms that an OR gate can be successfully implemented using only NAND gates by applying De Morgan's theorem.

2.10 Discussion

The NAND gate is a universal logic gate capable of implementing any Boolean function. In this experiment, the OR gate was realized by first generating the complements of the two input signals using NAND gates configured as inverters. These complemented signals were then applied to a third NAND gate, producing the OR function according to De Morgan's theorem.

The use of Logisim Evolution allowed the circuit to be designed, simulated, and verified without physical hardware. The simulation demonstrated that the implemented circuit behaved identically to a standard OR gate for every possible input combination. This experiment illustrates the practical importance of universal gates in digital logic design, where a single gate type can be used to construct complex digital systems.

2.11 Conclusion

In this experiment, an OR gate was successfully implemented using only NAND gates in the Logisim Evolution simulation environment. The simulation results agreed completely with the theoretical OR gate truth table, confirming the correctness of the design. The experiment demonstrated the application of De Morgan's theorem and highlighted the significance of NAND gates as universal building blocks in digital electronics.

3 Experiment 3: Implementation of AND Gate Using NOR Gates

3.1 Objective

The objective of this experiment is to implement a two-input AND gate using only NOR gates in the Logisim Evolution simulation environment. This experiment aims to demonstrate the universality of the NOR gate and verify that an AND gate can be realized exclusively using NOR logic.

3.2 Theory

Digital logic circuits are constructed using logic gates that perform Boolean operations on binary inputs. Among the basic logic gates, the AND gate produces a logic HIGH output only when all of its inputs are HIGH.

The Boolean expression of a two-input AND gate is

$$Y = A \cdot B \quad (12)$$

where

- A and B are the input variables.
- Y is the output.

The truth table of a two-input AND gate is presented in Table 6.

Table 6: Truth Table of a Two-Input AND Gate

Input A	Input B	Output ($A \cdot B$)
0	0	0
0	1	0
1	0	0
1	1	1

The NOR gate is known as a universal logic gate because every Boolean function can be implemented using only NOR gates. The Boolean expression of a NOR gate is

$$Y = \overline{A + B} \quad (13)$$

Using De Morgan's theorem, an AND gate can be implemented entirely with NOR gates as

$$A \cdot B = \overline{\overline{A} + \overline{B}} \quad (14)$$

where

$$\overline{A} = A \downarrow A \quad (15)$$

$$\overline{B} = B \downarrow B \quad (16)$$

The complete implementation therefore requires three NOR gates:

1. One NOR gate to generate $\overline{A}$.
2. One NOR gate to generate $\overline{B}$.
3. One NOR gate to perform $\overline{\overline{A} + \overline{B}}$, producing the AND output.

3.3 Required Software

- Logisim Evolution

3.4 Circuit Diagram

The AND gate was implemented using three NOR gates in Logisim Evolution.

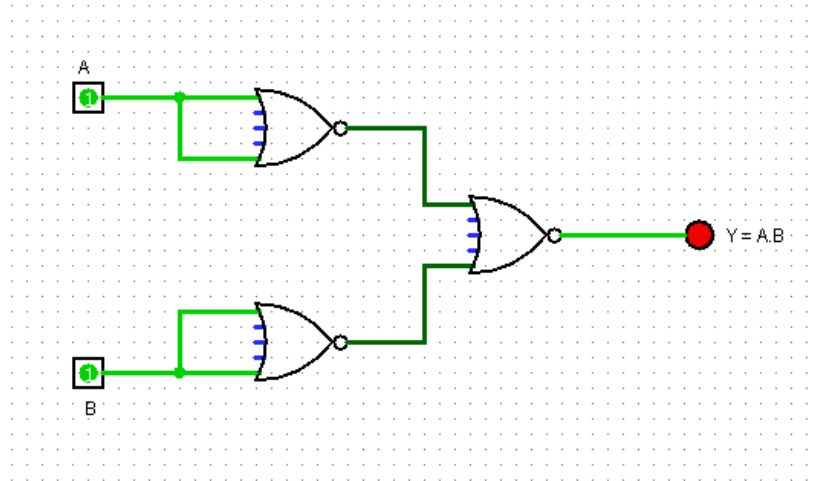


Figure 3: Implementation of AND Gate Using NOR Gates in Logisim Evolution

3.5 Simulation Procedure

The experiment was carried out entirely through simulation using Logisim Evolution. No physical hardware components were used.

The following steps were performed:

1. Open Logisim Evolution and create a new project.
2. Insert three two-input NOR gates from the Gates library.
3. Place two input pins representing the input variables A and B .
4. Connect input A to both inputs of the first NOR gate to obtain the complemented signal $\overline{A}$.
5. Connect input B to both inputs of the second NOR gate to obtain the complemented signal $\overline{B}$.

6. Connect the outputs of the first and second NOR gates to the inputs of the third NOR gate.
7. Connect an output pin to the output of the third NOR gate.
8. Apply all four possible input combinations using the input pins.
9. Observe the output generated by the simulation.
10. Verify that the simulated output matches the standard truth table of a two-input AND gate.

3.6 Simulation Setup

The logic circuit was designed and verified using Logisim Evolution.

The simulation consisted of

- Two input pins (A and B)
- Three NOR gates
- One output pin
- Connecting wires

Default gate propagation settings provided by Logisim Evolution were used throughout the simulation.

The observed outputs are summarized in Table 7.

Table 7: Observed Simulation Results

Input A	Input B	Expected Output	Observed Output
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

3.7 Discussion

The simulation verified that an AND gate can be successfully realized using only NOR gates. Initially, the first two NOR gates were configured as inverters by connecting each gate's inputs together. These produced the

complemented inputs $\overline{A}$ and $\overline{B}$. The third NOR gate combined the complemented signals according to De Morgan's theorem to generate the final AND output.

The simulated outputs matched the theoretical truth table for every possible input combination, confirming the correctness of the implemented logic circuit. This experiment also demonstrated the practical significance of universal gates in digital circuit design, where a single gate type can be used to implement any Boolean function.

3.8 Result

The AND gate was successfully implemented using only NOR gates in the Logisim Evolution simulation environment. The simulation results agreed completely with the theoretical truth table of the AND gate for all input combinations.

3.9 Conclusion

This experiment demonstrated the implementation of a two-input AND gate using only NOR gates through digital simulation in Logisim Evolution. The results confirmed the universal property of the NOR gate and validated the application of De Morgan's theorem in digital logic design. The experiment enhanced understanding of Boolean algebra, logic gate transformation, and simulation-based verification of combinational logic circuits.

4 Experiment 4: Implementation of AND Gate Using NAND Gates

4.1 Objective

The objective of this experiment is to implement a two-input AND gate using only NAND gates in the Logisim Evolution simulation environment. The experiment also aims to verify the operation of the implemented circuit by comparing the simulated output with the theoretical truth table of an AND gate.

4.2 Theory

The AND gate is one of the fundamental logic gates used in digital electronics. It performs logical multiplication and produces a HIGH output only when

all of its inputs are HIGH. Otherwise, the output remains LOW.

The Boolean expression of a two-input AND gate is

$$Y = A \cdot B$$

where A and B are the input variables and Y is the output.

The truth table of a two-input AND gate is presented in Table 8.

Table 8: Truth Table of a Two-Input AND Gate

Input A	Input B	Output ($A \cdot B$)
0	0	0
0	1	0
1	0	0
1	1	1

The NAND gate is a universal logic gate because any Boolean function can be realized using only NAND gates. The Boolean expression of a NAND gate is

$$Y = (A \cdot B)'$$

To obtain the AND operation using NAND gates, the output of the first NAND gate is inverted using another NAND gate configured as an inverter. Since a NAND gate acts as a NOT gate when both of its inputs are connected together,

$$\overline{\overline{A \cdot B}} = A \cdot B$$

Therefore,

$$AND(A, B) = \overline{NAND(A, B)}$$

This implementation requires two NAND gates.

4.3 Required Software

- Logisim Evolution

4.4 Circuit Diagram

Figure 4 illustrates the implementation of an AND gate using two NAND gates in Logisim Evolution.

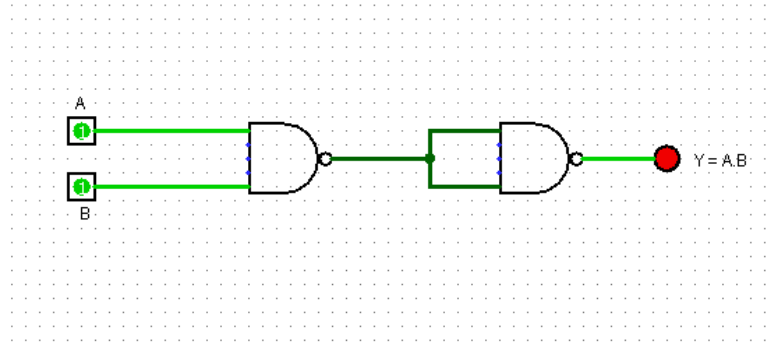


Figure 4: Implementation of AND Gate Using NAND Gates in Logisim Evolution

4.5 Simulation Procedure

The following steps were performed using Logisim Evolution:

1. Open Logisim Evolution and create a new circuit.
2. Place two input pins and one output pin from the Wiring library.
3. Insert two NAND gates from the Gates library.
4. Connect the two input pins (A and B) to the inputs of the first NAND gate.
5. Connect the output of the first NAND gate to both input terminals of the second NAND gate. This configuration allows the second NAND gate to function as an inverter.
6. Connect the output of the second NAND gate to the output pin.
7. Apply all four possible input combinations by toggling the input pins.
8. Observe and record the corresponding output for each input combination.
9. Compare the simulated outputs with the theoretical truth table of the AND gate.

4.6 Simulation Setup

The circuit was designed and simulated entirely using Logisim Evolution. Two input pins represented the logical variables A and B , while two NAND gates were interconnected to implement the AND function. An output pin displayed the resulting logic level. Input combinations were manually toggled to verify the behavior of the circuit.

4.7 Observed Truth Table

The simulation results are summarized in Table 9.

Table 9: Observed Simulation Results

Input A	Input B	Simulated Output
0	0	0
0	1	0
1	0	0
1	1	1

4.8 Discussion

The experiment successfully demonstrated the implementation of an AND gate using only NAND gates. The first NAND gate generated the complemented output of the logical AND operation. By feeding this output into both inputs of a second NAND gate, the complemented signal was inverted, producing the required AND function.

The simulation results matched the theoretical truth table for every possible input combination, confirming the correctness of the implemented design. This experiment illustrates the practical significance of universal gates and demonstrates that complex logic functions can be constructed using only NAND gates.

4.9 Result

The AND gate was successfully implemented using two NAND gates in the Logisim Evolution simulation environment. The simulated outputs were identical to the theoretical outputs for all input combinations, confirming the correct realization of the AND logic function.

4.10 Conclusion

The objective of implementing an AND gate using only NAND gates was successfully achieved through simulation in Logisim Evolution. The experiment verified that the NAND gate, as a universal gate, can be used to realize the AND operation by employing an additional NAND gate as an inverter. The observed simulation results were consistent with the theoretical truth table, reinforcing the concepts of Boolean algebra, universal gates, and combinational logic design.

4.11 Precautions

- Ensure that all gate connections are made correctly before running the simulation.
- Verify that the output of the first NAND gate is connected to both inputs of the second NAND gate.
- Check that all input pins are assigned valid binary logic values (0 or 1).
- Confirm that the simulation is active before observing the output.
- Compare the simulated output with the theoretical truth table to verify correctness.

5 Experiment 5: Implementation of NOT Gate Using NOR Gates

5.1 Objective

The objectives of this experiment are:

- To understand the logical operation of a NOT gate.
- To demonstrate that a NOR gate is a universal gate capable of implementing a NOT gate.
- To design and simulate a NOT gate using only NOR gates in Logisim Evolution.
- To verify the simulated output using the truth table.

5.2 Theory

A NOT gate, also known as an inverter, is one of the fundamental digital logic gates. It produces the logical complement of its input. If the input is logic HIGH (1), the output becomes logic LOW (0). Conversely, if the input is logic LOW (0), the output becomes logic HIGH (1).

The Boolean expression of a NOT gate is

$$Y = \overline{A}$$

where A is the input and Y is the inverted output.

A NOR gate is considered a universal logic gate because every basic logic gate can be implemented using only NOR gates. The logical expression of a two-input NOR gate is

$$Y = \overline{A + B}$$

When both inputs of the NOR gate are connected together, the expression becomes

$$Y = \overline{A + A}$$

Using the Idempotent Law,

$$A + A = A$$

Therefore,

$$Y = \overline{A}$$

which is exactly the Boolean expression of a NOT gate.

Thus, a single NOR gate with both of its inputs connected together behaves as a NOT gate.

5.3 Software Used

- Logisim Evolution (Digital Logic Circuit Simulator)

5.4 Circuit Diagram

Figure 5 illustrates the implementation of a NOT gate using a single NOR gate. Both input terminals of the NOR gate are connected to the same input signal.

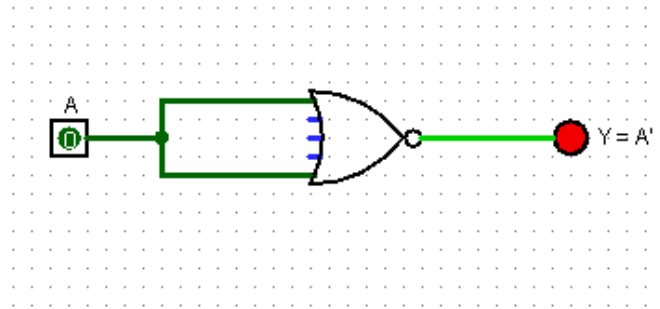


Figure 5: Implementation of a NOT Gate Using a NOR Gate in Logisim Evolution

5.5 Simulation Procedure

The experiment was carried out entirely using the Logisim Evolution simulation software.

1. Open Logisim Evolution and create a new circuit.
2. Insert one two-input NOR gate from the Gates library.
3. Connect the input signal A to both input terminals of the NOR gate.
4. Connect the output terminal of the NOR gate to an output pin labeled Y .
5. Place an input pin and use it to toggle between logic LOW (0) and logic HIGH (1).
6. Run the simulation by changing the input value.
7. Observe the output produced by the NOR gate.
8. Verify that the output is always the logical complement of the input according to the NOT gate truth table.
9. Capture the circuit diagram and simulation output for documentation.

5.6 Truth Table

Table 10: Truth Table of NOT Gate Implemented Using NOR Gate

Input (A)	Output ($Y = \bar{A}$)
0	1
1	0

5.7 Simulation Result

The simulation confirmed that the NOR gate successfully performed the inversion operation when both of its inputs were connected together. For every change in the input signal, the output produced the exact logical complement, which matches the expected behavior of a NOT gate.

5.8 Discussion

The experiment demonstrates one of the most important properties of universal gates. By connecting both inputs of a NOR gate to the same signal, the OR operation reduces to the input itself, and the final inversion performed by the NOR gate produces the NOT operation.

The simulation results obtained in Logisim Evolution were consistent with the theoretical Boolean expression and the truth table. This confirms that complex digital circuits can be designed using only NOR gates without requiring dedicated NOT gate components. Such implementations are fundamental in digital logic design and integrated circuit development.

5.9 Conclusion

In this experiment, a NOT gate was successfully implemented using a single NOR gate in the Logisim Evolution simulation environment. The observed simulation outputs matched the theoretical truth table for all possible input combinations. The experiment verified that the NOR gate can function as a NOT gate by connecting both of its inputs together, thereby reinforcing the concept of universal gates and their significance in digital logic circuit design.

6 Experiment 6: Implementation of NOT Gate Using NAND Gates

6.1 Objective

The objectives of this experiment are:

- To understand the logical operation of the NOT gate.
- To verify that a NAND gate can function as a universal gate.
- To implement a NOT gate using only a NAND gate in Logisim Evolution.
- To verify the truth table of the implemented circuit through digital simulation.

6.2 Theory

The NOT gate, also known as an inverter, is one of the fundamental digital logic gates. It produces the complement of its input signal. If the input is logic HIGH (1), the output becomes logic LOW (0), and if the input is logic LOW (0), the output becomes logic HIGH (1).

The Boolean expression of a NOT gate is

$$Y = \overline{A}$$

A NAND gate is a universal logic gate because every basic logic gate can be implemented using only NAND gates. A NOT gate can be realized by connecting both inputs of a NAND gate together so that they receive the same input signal.

The Boolean expression for implementing a NOT gate using a NAND gate is

$$Y = A \text{ NAND } A$$

Using the NAND operation,

$$Y = \overline{A \cdot A}$$

Since

$$A \cdot A = A$$

therefore,

$$Y = \overline{A}$$

which is identical to the Boolean expression of a NOT gate.

This experiment demonstrates one of the simplest applications of universal gates and verifies that a single NAND gate can perform the inversion operation.

6.3 Software Used

The circuit was designed and simulated using the following software:

- Logisim Evolution

6.4 Components Used

The following components available in Logisim Evolution were used for the simulation:

- NAND Gate
- Input Pin
- Output Pin
- Wiring Connections

6.5 Truth Table

Table 11: Truth Table of NOT Gate

Input (A)	Output ($Y = \overline{A}$)
0	1
1	0

6.6 Circuit Diagram

The NOT gate was implemented by connecting both input terminals of a NAND gate to the same input signal. The complete circuit was designed in Logisim Evolution.

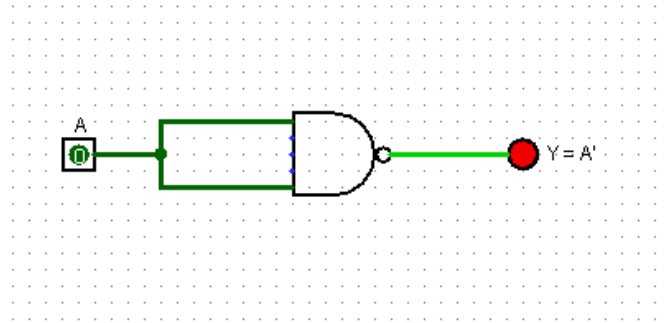


Figure 6: Implementation of NOT Gate Using NAND Gate in Logisim Evolution

6.7 Simulation Procedure

The following steps were performed in Logisim Evolution to simulate the circuit:

1. Open Logisim Evolution and create a new project.
2. Place one NAND gate from the Gates library onto the workspace.
3. Add one input pin and one output pin.
4. Connect the same input pin to both input terminals of the NAND gate.
5. Connect the output terminal of the NAND gate to the output pin.
6. Run the simulation by toggling the input pin between logic 0 and logic 1.
7. Observe and record the output produced for each input condition.
8. Compare the observed outputs with the theoretical truth table of the NOT gate.
9. Verify that the simulated circuit performs the inversion operation correctly.

6.8 Observation

The observed outputs obtained during simulation are shown in Table 12.

Table 12: Observed Simulation Results

Input (A)	Observed Output (Y)	Expected Output
0	1	1
1	0	0

6.9 Result

The Logisim Evolution simulation successfully demonstrated that a NOT gate can be implemented using a single NAND gate by connecting both of its input terminals together. The simulated outputs exactly matched the theoretical truth table of the NOT gate, confirming the correctness of the circuit implementation.

6.10 Discussion

The experiment illustrates one of the simplest applications of the universal NAND gate. Instead of using a dedicated inverter, the inversion operation can be achieved by connecting both inputs of a NAND gate to the same logic signal. During simulation in Logisim Evolution, the circuit behaved exactly as predicted by Boolean algebra. The observed outputs matched the expected logical inversion for every input combination without any discrepancies. This experiment reinforces the concept that universal gates can be used to construct all basic logic gates and forms the basis for designing more complex combinational and sequential digital circuits.

6.11 Conclusion

The implementation of a NOT gate using a NAND gate was successfully completed through simulation in Logisim Evolution. The experiment verified that connecting both inputs of a NAND gate to the same input signal produces the complement of that signal. The simulation results agreed with the theoretical truth table, demonstrating the universal property of NAND gates and enhancing the understanding of digital logic circuit design.

7 Experiment 7

7.1 Title

Implementation of Full Adder and Verification of Its Operation Using Logisim Evolution

7.2 Objective

The objectives of this experiment are:

- To understand the operating principle of a Full Adder circuit.
- To design and implement a Full Adder using basic logic gates in Logisim Evolution.
- To understand the relationship between Half Adders and a Full Adder.
- To verify the functionality of the designed circuit by applying all possible input combinations.
- To analyze the generated Sum and Carry outputs using simulation results.

7.3 Theory

A **Full Adder** is a combinational digital logic circuit that performs the addition of three one-bit binary inputs. These inputs consist of two operand bits (A and B) and one carry input (C_{in}) received from the previous stage of addition. The circuit generates two outputs:

- **Sum (S)**
- **Carry Output (C_{out})**

Unlike a Half Adder, which can only add two binary bits, a Full Adder includes an additional carry input, making it suitable for multi-bit binary addition. Consequently, Full Adders are widely used in arithmetic circuits such as ripple carry adders, carry look-ahead adders, Arithmetic Logic Units (ALUs), processors, and digital computing systems.

A Full Adder can be constructed using two Half Adders and one OR gate. The Boolean expressions of a Full Adder are:

$$S = A \oplus B \oplus C_{in} \quad (17)$$

$$C_{out} = AB + C_{in}(A \oplus B) \quad (18)$$

where $\oplus$ denotes the Exclusive-OR (XOR) operation. The corresponding truth table is shown in Table 13.

Table 13: Truth Table of Full Adder

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Required Software

- Logisim Evolution

Required Components

The following logic components available in Logisim Evolution were used:

- XOR Gates (2)
- AND Gates (2)
- OR Gate (1)
- Input Pins (A, B, C_{in})
- Output Pins (Sum, C_{out})
- Wires and Splitters (if necessary)

Simulation Procedure

Since this experiment was performed entirely using **Logisim Evolution**, no physical hardware implementation was carried out.

1. Open Logisim Evolution and create a new project.
2. Construct the first Half Adder using one XOR gate and one AND gate.
 - Inputs: A and B
 - Outputs:

- Intermediate Sum (S_1)
 - Carry (C_1)
3. Construct the second Half Adder.
 - Inputs: S_1 and C_{in}
 - Outputs:
 - Final Sum (S)
 - Carry (C_2)
 4. Connect the carry outputs C_1 and C_2 to an OR gate to obtain the final carry output (C_{out}).
 5. Verify that all logic gates are correctly interconnected.
 6. Save the circuit design.
 7. Simulate the circuit by applying every possible combination of the three input variables (A , B , and C_{in}).
 8. Observe the generated outputs (Sum and C_{out}) for each input combination.
 9. Compare the simulated outputs with the theoretical truth table to verify the correctness of the circuit.

7.4 Circuit Diagram

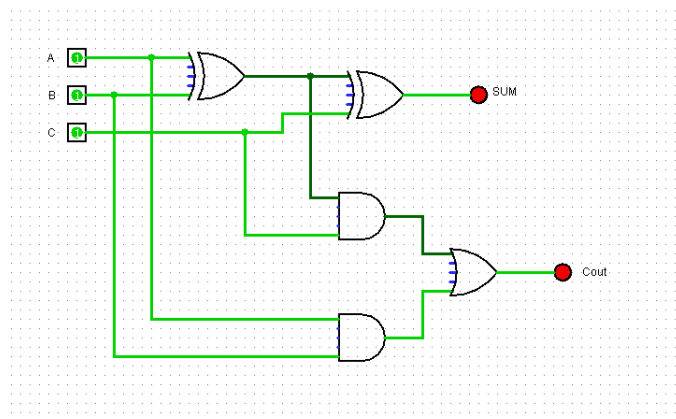


Figure 7: Full Adder Circuit Designed in Logisim Evolution

7.5 Observation

The simulated outputs were found to be consistent with the theoretical truth table of the Full Adder.

During simulation, several representative input combinations were examined. For example:

- When the 8-bit Full Adder displayed 10 at the Sum output and 0 at the Carry Output, the least significant bit corresponded to a one-bit addition with $A = 1$, $B = 1$, and $C_{in} = 0$, which produced $S = 0$ and $C_{out} = 1$. This agrees with the expected Full Adder operation.
- Similarly, when the 8-bit Full Adder displayed 11 at the Sum output and 0 at the Carry Output, the least significant stage corresponded to $A = 1$, $B = 1$, and $C_{in} = 1$, yielding $S = 1$ and $C_{out} = 1$, which also matches the theoretical result.

$$A = 1, B = 1, C_{in} = 0$$

which produced:

$$S = 0, \quad C_{out} = 1$$

This agrees with the expected Full Adder operation.

- Similarly, when the 8-bit Full Adder displayed 11 at the Sum output and 0 at the Carry Output, the least significant stage corresponded to:

$$A = 1, B = 1, C_{in} = 1$$

yielding:

$$S = 1, \quad C_{out} = 1$$

which also matches the theoretical result.

- All remaining input combinations produced outputs identical to those listed in the truth table, confirming the correctness of the designed circuit.

7.6 Discussion

The Full Adder was successfully implemented by combining two Half Adders with one OR gate. Simulation in Logisim Evolution demonstrated that the circuit correctly performed binary addition for every possible input combination.

The XOR gates generated the required Sum output, while the AND and OR gates correctly generated the Carry output. Since the experiment was performed entirely through software simulation, logic verification could be completed without requiring any physical electronic components.

The experiment also demonstrated that a Full Adder serves as the fundamental building block for constructing multi-bit binary adders. The successful operation of the simulated circuit confirms the correctness of the theoretical Boolean expressions and logical design.

7.7 Result

The Full Adder circuit was successfully designed and simulated in Logisim Evolution. The observed Sum and Carry outputs matched the theoretical truth table for all eight possible input combinations, confirming the correct implementation and operation of the Full Adder.

7.8 Conclusion

In this experiment, a Full Adder was designed and implemented using basic logic gates in Logisim Evolution. The circuit was constructed from two Half Adders and one OR gate, demonstrating how complex arithmetic circuits can be built from simpler digital components. Simulation results verified the correct generation of both the Sum and Carry outputs for every possible input combination.

This experiment strengthened the understanding of binary addition, combinational logic design, Boolean algebra, and hierarchical circuit implementation. Since Full Adders are the basic building blocks of multi-bit adders and Arithmetic Logic Units (ALUs), mastering their design is essential for the study of digital electronics and computer architecture.

8 Experiment 8

8.1 Title

Implementation of a Binary to BCD Converter Using Logisim Evolution

8.2 Objective

The objectives of this experiment are:

- To understand the concept of Binary-Coded Decimal (BCD) representation.
- To study the conversion of a 4-bit binary number into its corresponding BCD output.
- To design and simulate a Binary to BCD converter using Logisim Evolution.
- To verify the correctness of the converter by testing different binary input combinations.
- To understand the practical importance of Binary to BCD conversion in digital display systems.

8.3 Theory

Digital systems internally perform arithmetic and logical operations using binary numbers. However, many digital devices such as calculators, digital clocks, counters, and measuring instruments display information in decimal form for easy interpretation. Therefore, a conversion circuit is often required to convert binary numbers into their equivalent decimal representation.

Binary-Coded Decimal (BCD) is a numerical representation in which each decimal digit is represented by its corresponding 4-bit binary code. Unlike ordinary binary representation, each decimal digit is encoded separately. For example,

$$(25)_{10} = (0010\ 0101)_{BCD}$$

where the decimal digits 2 and 5 are individually represented by their 4-bit binary equivalents.

A Binary to BCD converter is a combinational logic circuit that converts a binary input into its equivalent BCD representation. For a 4-bit binary input, values from decimal 0 to 9 can be represented directly in BCD with a single BCD digit. Binary values from 10 to 15 require two BCD digits (tens and ones). During this experiment, the converter circuit was designed and simulated using basic logic gates available in Logisim Evolution.

The converter receives four binary input bits:

$$B_3, B_2, B_1, B_0$$

where B_3 is the Most Significant Bit (MSB) and B_0 is the Least Significant Bit (LSB).

The output consists of four BCD bits:

$$D_4, D_3, D_2, D_1$$

which represent the converted decimal digit and can be connected to a BCD-to-Seven-Segment Decoder for display purposes.

Binary to BCD conversion is widely used in:

- Digital clocks
- Electronic calculators
- Digital voltmeters
- Frequency counters
- Embedded systems with seven-segment displays
- Industrial control and measurement systems

Since the converter is a combinational circuit, its outputs depend only on the present input values and do not require any memory elements.

8.4 Required Software

- Logisim Evolution
- Computer with Windows/Linux operating system

8.5 Components Used

The following components available in Logisim Evolution were used to implement the circuit:

- Input Pins (4)
- AND Gates
- OR Gates
- NOT Gates
- XOR Gates (if required)
- Output Pins

- BCD to Seven-Segment Display Decoder
- Seven-Segment Display
- Wires and Splitters

8.6 Truth Table

Table 14 shows the Binary to BCD conversion for all possible 4-bit binary inputs.

Table 14: Truth Table of Binary to BCD Converter

B_3	B_2	B_1	B_0	D_4	D_3	D_2	D_1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1
0	1	1	0	0	1	1	0
0	1	1	1	0	1	1	1
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	1
1	0	1	0	1	0	1	0
1	0	1	1	1	0	1	1
1	1	0	0	1	1	0	0
1	1	0	1	1	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1

8.7 Circuit Diagram

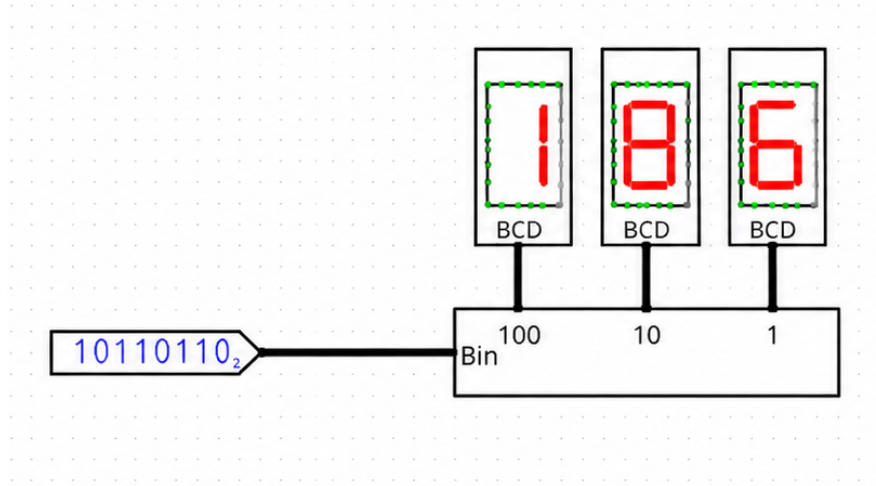


Figure 8: Binary to BCD Converter Circuit Implemented in Logisim Evolution

8.8 Simulation Setup

The Binary to BCD converter was designed entirely in Logisim Evolution using basic combinational logic gates. Four input pins represented the binary inputs (B_3 , B_2 , B_1 , and B_0), while the output pins represented the converted BCD outputs. The BCD outputs were further connected to a BCD-to-Seven-Segment Decoder and a Seven-Segment Display to visualize the converted decimal digit.

The simulation was performed by changing the binary input combinations using the poke tool available in Logisim Evolution. The corresponding BCD outputs and the displayed decimal digit were observed and compared with the expected theoretical values.

8.9 Procedure

The experiment was carried out through simulation in Logisim Evolution following the steps below:

1. Open Logisim Evolution and create a new project.
2. Place four input pins and label them as B_3 , B_2 , B_1 , and B_0 .

3. Design the Binary to BCD converter using the required logic gates according to the conversion logic.
4. Connect the generated BCD outputs to output pins labeled D_4 , D_3 , D_2 , and D_1 .
5. Connect the BCD outputs to a BCD-to-Seven-Segment Decoder and then to a Seven-Segment Display.
6. Verify all wiring connections and ensure there are no floating inputs or disconnected outputs.
7. Run the simulation using the poke tool.
8. Apply different combinations of binary inputs from 0000 to 1111.
9. Observe the corresponding BCD outputs and the decimal digit displayed on the seven-segment display.
10. Compare the observed outputs with the expected values given in the truth table.
11. Capture screenshots of the completed circuit and selected simulation results for documentation.

8.10 Observed Simulation Results

The Binary to BCD converter was successfully simulated in Logisim Evolution. Different binary input combinations were applied, and the corresponding BCD outputs were obtained correctly. The outputs matched the expected conversion values listed in the truth table.

The seven-segment display correctly represented the decimal equivalent corresponding to the generated BCD output. No incorrect outputs or unstable conditions were observed during the simulation.

8.11 Result

The Binary to BCD converter was successfully designed and simulated in Logisim Evolution. The converter accurately transformed each 4-bit binary input into its corresponding BCD representation. The observed outputs matched the theoretical truth table, confirming the correct operation of the implemented combinational logic circuit.

8.12 Discussion

The Binary to BCD converter demonstrates the practical application of combinational logic circuits in digital systems. Binary representation is efficient for computation inside digital devices, whereas decimal representation is more convenient for human interpretation. Therefore, Binary to BCD converters serve as an important interface between computational circuits and display systems.

Throughout the simulation, the converter correctly generated the expected BCD outputs for all tested binary inputs. The integration of a BCD-to-Seven-Segment Decoder allowed the converted values to be visualized clearly on the seven-segment display, making verification straightforward.

The experiment also reinforced the understanding that combinational circuits produce outputs solely based on the present input values without requiring memory or sequential elements. Simulation using Logisim Evolution provided an efficient platform for designing, testing, and validating the circuit before any hardware implementation.

8.13 Conclusion

In this experiment, a Binary to BCD converter was successfully implemented and verified using Logisim Evolution. The simulation demonstrated the correct conversion of binary inputs into their equivalent BCD outputs, which were subsequently displayed on a seven-segment display through a BCD decoder. The experiment enhanced the understanding of binary number systems, BCD representation, combinational logic design, and their practical applications in digital display devices. The simulation results were consistent with the theoretical truth table, confirming the correctness of the implemented circuit.

8.14 References

References

- [1] M. M. Mano and M. D. Ciletti, *Digital Design*, 6th ed., Pearson, 2018.
- [2] T. L. Floyd, *Digital Fundamentals*, 11th ed., Pearson, 2015.
- [3] R. J. Tocci, N. Widmer, and G. Moss, *Digital Systems: Principles and Applications*, 12th ed., Pearson, 2017.

- [4] R. H. Katz and G. Borriello, *Contemporary Logic Design*, 2nd ed., Pearson, 2005.
- [5] Logisim Evolution Developers, “Logisim Evolution: Digital Logic Simulator,” GitHub Repository. Available: <https://github.com/logisim-evolution/logisim-evolution> (Accessed: Jul. 2026).